Name: Spoorthi A

Course: Artificial Intelligence

Project No 1 – Sentiment Analysis

Project using CNN

Project - 1

Sentiment Analysis using CNN

Dataset: IMDB Movie Reviews | Framework: TensorFlow / Keras

Table of Contents

1. What is This Project About?

This project is about teaching a computer to read a movie review and decide whether it is positive or negative. For example, if someone writes 'This movie was amazing!', the computer should say Positive. If someone writes 'Worst film ever, totally boring', the computer should say Negative.

To do this, we use a method called CNN — Convolutional Neural Network. Even though CNNs are mostly known for image recognition, they also work very well for understanding text patterns. We train the CNN on 50,000 real movie reviews from IMDB, and after training, the model can classify new reviews on its own.

> We give the computer thousands of reviews that are already labelled (positive or negative). The CNN learns from them. After learning, when we give it a new review, it can predict whether it is positive or negative on its own.

2. Tools and Libraries Used

Before running the code, install the required libraries using this command:

```
pip install tensorflow keras numpy matplotlib
```

Library / Tool	What It Does
Python	The programming language we use to write the entire project
TensorFlow / Keras	Used to build and train the CNN model. Keras makes it simple with easy commands
NumPy	Helps with handling numbers and arrays efficiently
Matplotlib	Used to draw graphs — accuracy graph and loss graph
IMDB Dataset	A ready-made dataset of 50,000 movie reviews, already built into Keras

3. About the Dataset

I have used the IMDB Movie Reviews dataset. It contains 50,000 reviews written by real people about movies. Each review is already labelled as either Positive (1) or Negative (0). The dataset is split into two halves: 25,000 reviews for training and 25,000 for testing.

> **Why IMDB?**
> It is freely available inside Keras — no downloading needed. It is already converted into numbers so we can directly feed it into the model. It is balanced — exactly 50% positive and 50% negative reviews.

One important thing to understand is that the dataset does not store the actual words. Instead, every word is replaced with a number. For example, the word 'good' might be stored as 45, and the word 'bad' might be stored as 78. This is because computers work with numbers, not words.

Tokenization was not performed manually in this project because the IMDB dataset provided by Keras is already pre-tokenized. Each review is already represented as a sequence of integers where each integer corresponds to a word ranked by frequency. So, I directly applied padding on top of these pre-tokenized sequences to make them a uniform length of 200 tokens."

Detail	Value
Total Reviews	50,000
Training Reviews	25,000 (we train the model on these)
Testing Reviews	25,000 (we check accuracy on these)
Vocabulary Size Used	Top 10,000 most common words
Review Length (after padding)	200 words per review
Labels	1 = Positive Review, 0 = Negative Review

4. Step-by-Step Explanation of the Code

Step 1 — Import Libraries

First bring in all the tools needed. Think of this like gathering all your ingredients before cooking.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense,
Dropout
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

Step 2 — Set Basic Settings

Define a few numbers that control how the model works. Putting them all at the top makes it easy to change them later.

```
VOCAB_SIZE = 10000   # only use the 10,000 most common words
MAX_LEN    = 200     # every review will be exactly 200 words long
EPOCHS     = 10      # the model will learn from the data 10 times
BATCH_SIZE = 64      # process 64 reviews at a time during training
```

> **What is an Epoch?**
> One epoch means the model has seen all the training data once. With 10 epochs, the model goes through all 25,000 reviews 10 times, getting better each time — similar to practising a skill repeatedly.

Step 3 — Load the Dataset

Keras has the IMDB dataset built in. One line of code loads all 50,000 reviews, already split into training and testing sets.

```
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=VOCAB_SIZE)
```

Here, X_train contains the reviews (as numbers) and y_train contains the labels (0 or 1). The num_words=10000 means we only keep the 10,000 most common words and ignore very rare words.

Step 4 — Padding the Reviews

Reviews have different lengths. One review might be 80 words long and another might be 300 words long. The CNN needs all inputs to be the same size, so we pad every review to exactly 200 words.

```
X_train = pad_sequences(X_train, maxlen=MAX_LEN)
X_test  = pad_sequences(X_test,  maxlen=MAX_LEN)
```

What is Padding?
Padding means adding zeros to short reviews to make them 200 words long. If a review is 150 words, we add 50 zeros at the beginning. If a review is 250 words, we cut it to 200. This ensures every review entering the model is exactly the same size.

Step 5 — Building the CNN Model

This is the most important step. Building the model layer by layer using Sequential, which means stack layers one on top of the other in order.

```
model = Sequential()
```

Layer 1 — Embedding Layer

```
model.add(Embedding(input_dim=10000, output_dim=64, input_length=200))
```

Remember that words are stored as numbers (e.g., 'good' = 45). The Embedding layer converts each number into a list of 64 values that represent the meaning of that word. For example, words like 'good', 'great', and 'excellent' will have similar values because they mean similar things. This helps the model understand relationships between words.

Layer 2 — Conv1D Layer (The CNN Part)

```
model.add(Conv1D(filters=64, kernel_size=3, activation='relu'))
```

This is the core CNN layer. It scans the review 3 words at a time (kernel_size=3), looking for useful patterns. For example, it might learn that the pattern 'not good at' is negative, or 'really loved it' is positive. It does this with 64 different filters, each looking for different patterns. The activation='relu' simply means: if you find something useful, keep it; if not, ignore it.

Layer 3 — GlobalMaxPooling1D Layer

```
model.add(GlobalMaxPooling1D())
```

After the Conv1D layer scans the entire review, we have many values. GlobalMaxPooling1D picks the single highest (most important) value from each filter. This reduces a lot of data into a small, meaningful summary. Think of it as: from everything the filter found, what was the strongest signal?

Layer 4 — Dense Layer

```
model.add(Dense(64, activation='relu'))
```

This is a regular fully connected layer. It takes the summarised features from the previous step and learns more complex combinations. It has 64 neurons, each looking at all the inputs and contributing to the final answer.

Layer 5 — Dropout Layer

```
model.add(Dropout(0.5))
```

Dropout randomly switches off 50% of the neurons during each training step. This sounds strange but it is a very effective technique. It forces the model not to rely too much on any single neuron, which prevents overfitting. Overfitting is when the model memorises the training data but fails on new data.

Layer 6 — Output Layer

```
model.add(Dense(1, activation='sigmoid'))
```

The final layer has just 1 neuron. It outputs a single number between 0 and 1. If the output is above 0.5, the review is classified as Positive. If below 0.5, it is Negative. The sigmoid activation is used here because it naturally outputs values between 0 and 1, making it perfect for binary (yes/no) classification.

Step 6 — Compiling the Model

Before training, we need to tell the model three things: how to update its weights (optimizer), what to measure as error (loss), and what to track (metrics).

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Setting	Simple Explanation
adam	A smart optimizer that automatically adjusts how fast the model learns
binary_crossentropy	The error function for problems with two outcomes (positive or negative)
accuracy	We want to track what percentage of reviews the model gets correct

Step 7 — Training the Model

Actual training of the model. Showing it the training reviews and let it learn.

```
history = model.fit(X_train, y_train, epochs=10, batch_size=64,
validation_split=0.1)
```

During training, the model sees 64 reviews at a time (batch_size=64), calculates how wrong it was, and adjusts its weights to do better. It does this for all 25,000 training reviews, and repeats the whole process 10 times (epochs=10). The validation_split=0.1 means 10% of the training data is set aside to check progress after each epoch without using it for learning.

Step 8 — Evaluating the Model

After training is done, test the model on the 25,000 test reviews it has never seen before.

```
loss, accuracy = model.evaluate(X_test, y_test)
print('Test Accuracy:', round(accuracy * 100, 2), '%')
```

Expected Result

This simple CNN model typically achieves around 88% to 90% accuracy on the IMDB test set. This means it correctly classifies about 9 out of every 10 movie reviews as positive or negative.

Step 9 — Plotting the Results

Draw two graphs to visually understand how the model trained over the 10 epochs.

```
plt.plot(history.history['accuracy'],     label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
```

The accuracy graph shows how accurate the model was after each epoch on both training data and validation data. Ideally both lines go up together. The loss graph shows how much error the model had — ideally both lines go down together. If the training accuracy is much higher than validation accuracy, it means the model is overfitting.

Step 10 — Testing on New Reviews

Finally, test the model on completely new sentences that has been written by ourselves.

```
predict_sentiment('This movie was amazing! I really loved it.')
predict_sentiment('Terrible film. Boring and a complete waste of time.')
```

The function converts our sentence into numbers, pads it to 200 words, feeds it into the trained model, and prints whether it is POSITIVE or NEGATIVE along with a confidence score.

5. Model Summary

Here is a simple overview of all 6 layers in our CNN model and what each one does:

SL.NO	Layer	What It Does
1	**Embedding**	Converts word numbers into meaningful vectors (lists of 64 values)
2	**Conv1D**	Scans 3 words at a time to find useful patterns (like phrases)
3	**GlobalMaxPooling1D**	Picks the most important feature found by each filter
4	**Dense (64)**	Learns complex combinations of the features found so far
5	**Dropout (0.5)**	Randomly turns off 50% of neurons to prevent overfitting
6	**Dense (1)**	Outputs a single number: above 0.5 = Positive, below = Negative

6. Results

Accuracy

After training for 10 epochs, the model achieves around 88% to 90% accuracy on the test set. This means if we give it 100 reviews it has never seen before, it correctly classifies around 88 to 90 of them.

Metric	Approximate Result
Test Accuracy	~88% to 90%
Test Loss	~0.27 to 0.30
Training Time	~5 to 10 minutes on CPU
Model Stops At	Around epoch 7 to 10

Sample Predictions

Review	Prediction
This movie was amazing! I really loved it.	**POSITIVE**
Terrible film. Boring and a complete waste of time.	**NEGATIVE**
It was okay, nothing special but not bad either.	**POSITIVE**

7. Confusion Matrix and Classification Report

After training is complete, accuracy alone is not sufficient. A Confusion Matrix shows us exactly how many reviews the model got right and how many it got wrong, broken down by category. For example, it tells us: how many negative reviews were correctly identified, and how many were mistakenly called positive.

The Code Used

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative','Positive'],
yticklabels=['Negative','Positive'])
print(classification_report(y_test, y_pred,
target_names=['Negative','Positive']))
```

Line-by-Line Explanation

- **model.predict(X_test)** — The trained model looks at all 25,000 test reviews and gives a score between 0 and 1 for each one. A score close to 1 means it thinks the review is Positive. A score close to 0 means Negative.
- **(y_pred_prob >= 0.5).astype(int)** — Convert the score into a hard decision: if the score is 0.5 or above, we call it 1 (Positive). If below 0.5, we call it 0 (Negative). This gives us a clean list of 0s and 1s to compare against the real labels.
- **confusion_matrix(y_test, y_pred)** — Compares the model's predictions (y_pred) with the actual correct answers (y_test) and builds a 2x2 table showing four counts: correct positives, correct negatives, and the two types of mistakes.
- **sns.heatmap(...)** — Draws the confusion matrix as a colour-coded grid (heatmap) using the seaborn library. Darker blue means a higher number. The annot=True shows the actual counts inside each box, and fmt='d' means display them as whole numbers.
- **classification_report(...)** — Prints a detailed text report showing Precision, Recall, and F1-Score for both Negative and Positive classes separately. This gives a much more detailed view of performance than a single accuracy number.

How to Read the Confusion Matrix

The confusion matrix is a 2x2 table. Each box tells us something different about how the model performed:

Box	Name	What it Means in Simple Words
Top-Left	True Negative (TN)	Review was Negative AND model correctly said Negative. Good!
Top-Right	False Positive (FP)	Review was Negative BUT model wrongly said Positive. Mistake!
Bottom-Left	False Negative (FN)	Review was Positive BUT model wrongly said Negative. Mistake!
Bottom-Right	True Positive (TP)	Review was Positive AND model correctly said Positive. Good!

How to Read the Classification Report ?

The classification_report prints three scores for each class. Here is what each one means :

- **Precision** — Out of all the reviews the model called Positive, how many were actually Positive? High precision means it does not make many false decision.
- **Recall** — Out of all the reviews that were actually Positive, how many did the model correctly find? High recall means it does not miss many positives.
- **F1-Score** — The average of Precision and Recall combined into one number. This is the most useful single number to judge overall performance for each class.

For a well-trained CNN on IMDB, Precision, Recall, and F1-Score all around 0.88 to 0.91 for both Negative and Positive classes. The diagonal boxes (top-left and bottom-right) of the confusion matrix will have the highest numbers, meaning most predictions were correct.

8. Challenges Faced

- **Reviews have different lengths:** Different reviews have very different word counts. I solved this using padding to make every review exactly 200 words long.
- **Overfitting:** The model was memorizing training data and performing poorly on new data. I added a Dropout layer (rate=0.5) to fix this.
- **Choosing the right settings:** Deciding on vocab size, sequence length, and number of filters required trial and error. The values I chose (10,000 vocab, 200 length, 64 filters) gave a good balance between speed and accuracy.
- **Training time:** On a CPU, each epoch takes 1 to 2 minutes. Running 10 epochs therefore takes about 10 to 20 minutes total.

9. Insights

1. CNN works well for text, not just images : Even though CNNs are mainly known for image processing, they perform very well on text too. The Conv1D layer scans words like a sliding window and picks up meaningful phrases , this project proves that CNN is a strong choice for sentiment analysis.

2. Balanced dataset gives fair results : The IMDB dataset has exactly 50% positive and 50% negative reviews. This is why the model performs equally well on both classes — it didn't get biased toward one side.

3. Dropout genuinely prevents overfitting : Without dropout, the training accuracy goes very high but test accuracy stays low , meaning the model memorized the data. With dropout at 0.5, both training and test accuracy stay close together, which is a sign of a healthy, generalizing model.

4. The confusion matrix reveals where the model struggles : Looking at the confusion matrix, the false positives and false negatives are roughly equal , meaning the model makes a similar number of mistakes on both classes. It doesn't have a specific weakness toward one type of error.

5. Word frequency matters more than rare words : By limiting vocabulary to the top 10,000 words, I removed very rare words that appear in few reviews. This actually improved performance because rare words add noise and confuse the model.

10. Conclusion

In this project, I successfully built a CNN model that can read a movie review and decide whether it is positive or negative. I used the IMDB dataset which has 50,000 labelled reviews, trained a 6-layer CNN model, and achieved around 88 to 90% accuracy on reviews the model had never seen before.

The key idea is that the CNN learns patterns in text , just like it learns patterns in images. The Conv1D layer scans the text, the Pooling layer picks the strongest signals, and the Dense layers make the final decision. The entire pipeline from loading data to making predictions is handled in a clear, step-by-step manner that can be easily extended to other text classification tasks.

11. References

[1] Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP 2014. arXiv:1408.5882

[2] Maas, A. L. et al. (2011). Learning Word Vectors for Sentiment Analysis. ACL 2011, pp. 142–150

[3] Chollet, F. et al. (2015). Keras: Deep Learning for Humans. https://keras.io

[4] Abadi, M. et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. OSDI 2016, pp. 265–283

[5] Srivastava, N. et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR, 15(1), 1929–1958